

MOHAMMED Fahad

SIO1

18/11/2025

Exercice 1 : La Classe Livre

Exercice 1.1 : Définir les attributs de la classe Livre

```
public class Livre {  
    private String titre;  
    private String auteur;  
    private double prix;
```

Les attributs sont les informations qu'on veut garder pour chaque livre. Ici on a besoin du titre, de l'auteur et du prix. On met "private" pour protéger ces informations.

Exercice 1.2 : Le constructeur

```
    public void Livre(String titre, String auteur, double prix)  
    {  
        this.titre = titre;  
        this.auteur = auteur;  
        this.prix = prix;
```

Le constructeur est une méthode qui crée un nouveau livre. Quand on écrit **new Livre("Programmer en C", "Claude Delannoy", 350)**, le constructeur prend ces 3 valeurs et les met dans les attributs du livre. Le mot "this" veut dire "ce livre-ci", donc **this.titre = titre** signifie qu'on met la valeur reçue dans l'attribut titre du livre.

Exercice 1.3 : Les getters et setters

```
        public String getTitre()  
        {  
            return titre;  
        }  
  
        public String getAuteur()  
        {  
            return auteur;  
        }  
  
        public double getPrix()  
        {  
            return prix;  
        }
```

Les getters permettent de lire les informations du livre. Par exemple `getTitre()` renvoie le titre du livre. Les setters permettent de modifier ces informations. Par exemple `setTitre("Nouveau titre")` change le titre du livre. Comme les attributs sont privés, on doit obligatoirement passer par ces méthodes pour accéder aux informations.

Exercice 1.3 : Les méthodes d'accès aux attributs

```
public void setTitre(String titre)
{
    this.titre = titre;
}

public void setAuteur(String auteur)
{
    this.auteur = auteur;
}

public void setPrix(double prix)
{
    this.prix = prix;
}
```

Les méthodes d'accès permettent d'accéder aux attributs privés de la classe. Il y a deux types : les méthodes "get" pour lire les informations (getTitre renvoie le titre) et les méthodes "set" pour modifier les informations (setTitre change le titre). Comme les attributs sont privés, on doit obligatoirement passer par ces méthodes pour les lire ou les modifier.

Exercice 1.4 : La méthode afficher()

```
public void afficher()
{
    System.out.println("Titre: " + titre + ", Auteur: " + auteur + ", Prix: " + prix);
}
}
```

La méthode afficher() permet d'afficher toutes les informations du livre en une seule ligne. Au lieu d'utiliser plusieurs getters pour afficher le titre, l'auteur et le prix séparément, on peut simplement appeler livre1.afficher() et tout s'affiche automatiquement. C'est plus rapide et pratique pour voir les informations complètes d'un livre.

Exercice 1.5 : Programme testant la classe Livre

```
package courcyber;
public class testLivre
{
    public static void main(String[] args)
    {
        System.out.println("Livre 1:");
        System.out.println("Donner le titre: Programmer en C");
        System.out.println("Donner l'auteur: Claude Delannoy");
        System.out.println("Donner le prix 350");

        Livre livre1 = new Livre("Programmer en C", "Claude Delannoy", 350);

        System.out.println("Le titre est " + livre1.getTitre());
        System.out.println("L'auteur est " + livre1.getAuteur());
        System.out.println("Le prix est " + livre1.getPrix());
        livre1.afficher();

        System.out.println("\nLivre 2:");
        System.out.println("Donner le titre: Programmer en Java");
        System.out.println("Donner l'auteur: Claude Delannoy");
        System.out.println("Donner le prix 450");

        Livre livre2 = new Livre("Programmer en Java", "Claude Delannoy", 450);

        livre2.afficher();
    }
}
```

Le programme de test permet de vérifier que la classe Livre fonctionne bien. On crée deux livres différents en utilisant le constructeur avec `new Livre(...)`. Ensuite, on utilise les `get` (`getTitre`, `getAuteur`, `getPrix`) pour afficher les informations de chaque livre une par une. On utilise aussi la méthode `afficher()` qui montre toutes les informations en une seule ligne. Grâce à ce programme, on peut voir que notre classe Livre marche correctement et qu'on peut créer plusieurs livres avec des informations différentes.

Exercice 2 : Classe compte

Exercice 2.1 : Les attributs de la classe Client

```
package courcyber;
public class Client
{
    private String CIN;
    private String nom;
    private String prenom;
    private String tel;
}
```

Les attributs sont les informations qu'on veut garder pour chaque client. Un client a besoin d'un CIN (numéro d'identité), un nom, un prénom et un numéro de téléphone. On utilise "private" pour protéger ces informations et empêcher qu'elles soient modifiées directement de l'extérieur.

Exercice 2.2 : Les méthodes d'accès aux attributs

```
public String getCIN()
{
    return CIN;
}

public void setCIN(String CIN)
{
    this.CIN = CIN;
}

public String getNom()
{
    return nom;
}

public void setNom(String nom)
{
    this.nom = nom;
}

public String getPrenom()
{
    return prenom;
}

public void setPrenom(String prenom)
{
    this.prenom = prenom;
}
```

```

    }

    public String getTel()
    {
        return tel;
    }

    public void setTel(String tel)
    {
        this.tel = tel;
    }

```

Les méthodes d'accès permettent d'accéder aux attributs privés de la classe Client. Il y a deux types : les méthodes "get" pour lire les informations du client (getCIN renvoie le CIN, getNom renvoie le nom) et les méthodes "set" pour modifier ces informations (setCIN change le CIN, setNom change le nom). Comme les attributs sont privés, on doit obligatoirement passer par ces méthodes pour les lire ou les modifier. Pour on crée ces méthodes pour tous les attributs : CIN, nom, prénom et tel.

Exercice 2.3 : Constructeur avec tous les attributs

```

// 3. Constructeur per mettant d'initialiser tous les attributs
public Client(String CIN, String nom, String prenom, String tel)
{
    this.CIN = CIN;
    this.nom = nom;
    this.prenom = prenom;
    this.tel = tel;
}

```

Ce constructeur permet de créer un client en donnant toutes ses informations. Le constructeur prend ces 4 valeurs et crée un client complet avec toutes ses informations.

Exercice 2.4 : Constructeur avec CIN, nom et prénom

```

public Client(String CIN, String nom, String prenom)
{
    this.CIN = CIN;
    this.nom = nom;
    this.prenom = prenom;
}

```

Ce constructeur permet de créer un client sans donner le numéro de téléphone. Parfois on ne connaît pas le téléphone d'un client, donc on peut créer le client avec seulement le CIN, le nom et le prénom. On pourra ajouter le téléphone plus tard avec le setter setTel().

Exercice 2.5 : La méthode afficher()

```
public void afficher()
{
    System.out.println("CIN: " + CIN);
    System.out.println("NOM: " + nom);
    System.out.println("Prénom: " + prenom);
    System.out.println("Tél : " + tel);
}
```

Cette méthode affiche toutes les informations du client ligne par ligne. Au lieu d'utiliser plusieurs getters pour afficher chaque information séparément, on peut simplement appeler `client1.afficher()` et toutes les informations s'affichent automatiquement. C'est plus pratique pour voir les détails d'un client.

Exercice 2.6 : La classe Compte

```
public class compte_6 {
    // 6. Définir les attributs
    private double solde;
    private int code;
    private static int compteur = 0;
    private Client proprietaire;

    // 6. Constructeur
    public void Compte(Client proprietaire)
    {
        compteur++;
        this.code = compteur;
        this.solde = 0;
        this.proprietaire = proprietaire;
    }
}
```

La classe `Compte` représente un compte bancaire. Elle a quatre attributs : le solde qui est l'argent dans le compte, le code qui est le numéro du compte, le compteur qui permet de donner un numéro unique à chaque nouveau compte, et le propriétaire qui est le client à qui appartient le compte. Quand on crée un nouveau compte, le constructeur prend un client en paramètre. Il augmente automatiquement le compteur de 1, donne ce numéro comme code au compte, met le solde à 0 (le compte est vide au départ), et enregistre le client comme propriétaire. Comme ça, chaque compte a un numéro différent qui s'incrémente automatiquement.

Exercice 2.7 : Les méthodes d'accès

```
public int getCode()
{
    return code;
}

public double getSolde()
{
    return solde;
}

public Client getProprietaire()
{
    return proprietaire;
}

public void setProprietaire(Client proprietaire)
{
    this.proprietaire = proprietaire;
}
}
```

Les méthodes d'accès permettent de lire les informations du compte. Pour le numéro de compte et le solde, on crée seulement les méthodes "get" (getCode et getSolde) sans les méthodes "set". Cela veut dire qu'on peut lire ces informations mais pas les modifier directement, c'est la lecture seule. Pour le propriétaire, on crée le getter (getProprietaire) pour lire et le setter (setProprietaire) pour modifier car on peut changer le propriétaire d'un compte. Cette protection empêche de changer le numéro de compte ou le solde n'importe comment.

Exercice 2.8 : Le constructeur

```
public void Compte(Client proprietaire)
{
    compteur++;
    this.code = compteur;
    this.solde = 0;
    this.proprietaire = proprietaire;
}
```

Ce constructeur permet de créer un compte en donnant juste le propriétaire du compte. Quand on écrit `new Compte(client1)`, le constructeur crée automatiquement un nouveau compte avec un numéro unique (en augmentant le compteur), met le solde à 0, et associe le client comme propriétaire. On n'a pas besoin de donner le code ou le solde car ils sont générés et initialisés automatiquement.

Exercice 2.9 : Les méthodes de la classe Compte

```
public void crediter(double somme)
{
    this.solde = this.solde + somme;
}
public void debiter(double somme)
{
    this.solde = this.solde - somme;
}
public void afficher()
{
    System.out.println("Numéro de Compte: " + code);
    System.out.println("Solde de compte: " + solde);
    System.out.println("Propriétaire du compte :");
    proprietaire.afficher();
}
}
```

Les méthodes de la classe Compte permettent de faire des opérations bancaires. La méthode `crediter(somme)` ajoute de l'argent au compte en augmentant le solde. La méthode `debiter(somme)` retire de l'argent du compte en diminuant le solde. Il y a aussi des versions avec deux paramètres : `crediter(somme, compte)` qui ajoute de l'argent au compte actuel et retire la même somme d'un autre compte, et `debiter(somme, compte)` qui fait l'inverse. La méthode `afficher()` montre toutes les informations du compte : son numéro, son solde et les informations de son propriétaire.

Exercice 2.10 : Programme de test pour la classe Compte

```
public class TestCompte
{
    public static void main(String[] args)
    {
        // Créer un client
        System.out.println("Compte 1:");
        System.out.println("Donner Le CIN: EE111222");
        System.out.println("Donner Le Nom: Salim");
        System.out.println("Donner Le Prénom: Omar");
        System.out.println("Donner Le numéro de téléphone: 0611111");

        Client client1 = new Client("EE111222", "Salim", "Omar", "0611111");
    }
}
```

```

        // Créer un compte pour ce client
        Compte compte1 = new Compte(client1);

        // Afficher les détails du compte
        System.out.println("\nDétails du compte:");
        compte1.afficher();

        // Déposer de l'argent
        System.out.println("\nDonner le montant à déposer: 5000");
        compte1.crediter(5000);
        System.out.println("Opération bien effectuée");

        // Afficher après dépôt
        System.out.println();
        compte1.afficher();

        // Retirer de l'argent
        System.out.println("\nDonner le montant à retirer: 1000");
        compte1.debiter(1000);
        System.out.println("Opération bien effectuée");

        // Afficher après retrait
        System.out.println();
        compte1.afficher();
    }
}

```

Le programme de test permet de vérifier que les classes Client et Compte fonctionnent bien ensemble. On commence par créer un client avec toutes ses informations (CIN, nom, prénom, téléphone). Ensuite on crée un compte bancaire pour ce client. Le programme affiche les détails du compte qui a un solde de 0 au départ. On utilise la méthode `crediter()` pour déposer 5000 dans le compte, puis on affiche à nouveau pour voir que le solde a augmenté. Enfin, on utilise la méthode `debiter()` pour retirer 1000 du compte, et on affiche une dernière fois pour voir que le solde a diminué. Ce programme démontre que toutes les fonctionnalités de notre compte bancaire marchent correctement.

Et voici le resultat,

Compte 1:

Donner Le CIN: EE111222

Donner Le Nom: Salim

Donner Le Prénom: Omar

Donner Le numéro de téléphone: 0611111

Détails du compte:

Numéro de Compte: 1

Solde de compte: 0.0

Propriétaire du compte :

CIN: EE111222

NOM: Salim

Prénom: Omar

Tél : 0611111

Donner le montant à déposer: 5000

Opération bien effectuée

Numéro de Compte: 1

Solde de compte: 5000.0

Propriétaire du compte :

CIN: EE111222

NOM: Salim

Prénom: Omar

Tél : 0611111

Donner le montant à retirer: 1000

Opération bien effectuée

Numéro de Compte: 1

Solde de compte: 4000.0

Propriétaire du compte :

CIN: EE111222

NOM: Salim

Prénom: Omar

Tél : 0611111

CONCLUSION

Dans ce TP, on a appris à créer des classes en Java pour représenter des objets de la vie réelle. Dans l'exercice 1, on a créé une classe Livre avec ses informations (titre, auteur, prix) et ses méthodes pour les manipuler. Dans l'exercice 2, on a créé deux classes Client et Compte qui travaillent ensemble pour simuler un compte bancaire. On a compris l'importance des attributs privés pour protéger les données, des constructeurs pour créer des objets, des getters et setters pour accéder aux informations, et des méthodes pour faire des actions. Ces exercices nous montrent les bases de la programmation orientée objet : créer un modèle (la classe) et ensuite créer plusieurs objets différents avec ce modèle. C'est une façon organisée et logique de programmer qui ressemble à la vie réelle.